

AGAI-03 / ASSIGNMENT REPORT

Vizcom

RAG Chatbot

A Hybrid Retrieval System for Product Documentation

Author

Janda van Dyk

Role

Product Expert / Solutions Architect at Vizcom

Repository

github.com/jandavdyk

Submitted

29 May 2026

Contents

1. Executive Summary
2. Introduction and Motivation
3. Architecture Overview
4. Implementation
 - 4.1 Phase 1 - Web Scraping
 - 4.2 Phase 2 - Synthetic Q&A Generation
 - 4.3 Phase 3 - Vector Database Construction
 - 4.4 Phase 4 - Hybrid Retrieval Logic
 - 4.5 Phase 5 - Streamlit User Interface
5. Testing and Results
6. Limitations and Future Improvements
7. Key Learnings
8. Conclusion

1. Executive Summary

This project implements a Retrieval-Augmented Generation (RAG) chatbot grounded in the public content of vizcom.com. The system answers natural-language questions about Vizcom by combining a curated synthetic question-answer dataset with semantic vector search over chunked web content, all coordinated by a Llama 3.3 70B language model running on the Groq inference platform.

The chatbot was built end to end over a two-week period and demonstrates the full lifecycle of a small production-style RAG system: ethical web scraping, synthetic data generation, vector indexing, hybrid retrieval with confidence-based fallbacks, conversation memory, query rewriting for pronoun resolution, and a polished Streamlit interface with source citations and graceful out-of-knowledge handling.

Forty pages scraped from vizcom.com, including all major product solutions pages.

Three hundred and twenty synthetic question-answer pairs generated with Llama 3.3 70B.

Two hundred and forty-four embedded chunks stored in a persistent Chroma vector store.

Three distinct retrieval modes (direct Q&A match, vector + LLM generation, graceful decline).

2. Introduction and Motivation

Why Vizcom

I currently work at Vizcom as a Product Expert and Solutions Architect. Vizcom is an AI-powered design platform used by product designers and creative teams to turn early sketches into high-fidelity renders, explore variations, and accelerate the path from concept to manufacture. My day-to-day work involves helping designers understand and use Vizcom in their daily design work. I am interested in moving closer to the product side of Vizcom and I am motivated to design internal tools as well as prototype new features and set up PR's using the Vizcom codebase.

That dual perspective shaped the project. On one side, I wanted to demonstrate a real working RAG pipeline for the AGAI-03 coursework. On the other, I wanted to explore how non-engineers on the Vizcom team could build internal AI tools (sales support, customer success FAQ assistants, design education aids) without waiting on engineering bandwidth. The chatbot built here is intentionally close to what an internal Vizcom tool might look like in v1.

My Background and Interest

I am passionate about creative technology in general and fashion technology specifically. The intersection of AI, design, and apparel is moving rapidly, and tools like Vizcom are at the center

of that shift. Building a chatbot that surfaces the company's own content - including its apparel and footwear solutions - felt like a meaningful, useful exercise rather than a generic homework assignment.

Project Scope

The assignment required: web scraping, synthetic Q&A generation, vector storage, a two-stage hybrid retrieval mechanism, and a Streamlit user interface. Beyond the rubric, I added LLM-based query rewriting (to handle conversational follow-ups) and a confidence-based guardrail that politely declines questions outside the knowledge base. These additions reflect lessons learned during testing rather than the original specification.

3. Architecture Overview

The system implements the canonical RAG pattern in five distinct phases, two of which run offline (data preparation) and three of which run at query time (retrieval, generation, presentation).

Pipeline at a Glance

- **Offline 1.** Crawl vizcom.com using its public sitemap, prioritizing solutions and product pages.
- **Offline 2.** Use Llama 3.3 70B to generate eight curated question-answer pairs per scraped page, yielding a synthetic Q&A dataset.
- **Offline 3.** Embed both the scraped chunks and the Q&A questions with sentence-transformers and store them in two separate Chroma collections.
- **Online 1.** On a user query, optionally rewrite the query (resolving pronouns against conversation history), then search the Q&A collection. If the top match is sufficiently close, return that curated answer.
- **Online 2.** If the Q&A search is too weak, fall back to vector search over the raw chunks, pass the retrieved context to the LLM, and synthesize an answer. If even vector search produces a weak match, decline gracefully.
- **Online 3.** Display the answer in a Streamlit chat interface with mode badges, confidence scores, source citations, and conversation memory.

Technology Stack

Language	Python 3.14
Web Scraping	requests + BeautifulSoup4 + lxml
LLM provider	Groq API
LLM model	Llama 3.3 70B (versatile)
Embedding model	sentence-transformers / all-MiniLM-L6-v2

Vector database	Chroma (persistent local store)
Document chunking	LangChain RecursiveCharacterTextSplitter
UI framework	Streamlit
Orchestration	LangChain (embeddings + text splitting)
Configuration	python-dotenv (.env secrets)

Why these specific choices

Groq + Llama 3.3 70B

Groq's inference speed is exceptional and their free tier comfortably covers a project of this size. Llama 3.3 70B offers quality competitive with proprietary frontier models for question-answer generation, which mattered because the Q&A dataset is the primary retrieval target.

Chroma over FAISS or Pinecone

The assignment recommended Chroma. It is also genuinely the right choice for a single-user local project: zero infrastructure cost, persistent storage out of the box, and a simple Python API. FAISS would be faster at scale but requires more setup; Pinecone is overkill for 244 chunks.

sentence-transformers / all-MiniLM-L6-v2

This embedding model is small (80MB), fast on CPU, and free. Its 384-dimensional embeddings are sufficient for the semantic granularity we need on a single-domain corpus. Larger models like mpnet would offer marginal quality gains at significantly higher compute cost.

Streamlit

Streamlit lets you build a credible chat interface in under a hundred lines of Python with full session state, caching, and rich widgets. For a non-engineering team interested in prototyping, this is the obvious lowest-friction choice.

4. Implementation

4.1 Phase 1 — Web Scraping

The scraper module (`src/scrapper.py`) fetches `vizcom.com/sitemap.xml`, parses the URL list, prioritizes core pages over blog posts, and saves clean text to `data/raw/`. Vizcom's `robots.txt` is fully permissive (User-agent: * Allow: /), so scraping was both ethically and technically straightforward.

Pipeline

- Fetch the sitemap with a polite User-Agent identifying the project as an educational assignment.
- Parse XML, handling both regular sitemaps and sitemap-index files.
- For each page, strip `<script>`, `<style>`, `<nav>`, `<footer>`, `<header>`, `<aside>` and `<form>` elements using BeautifulSoup.
- Prefer `<main>` or `<article>` content; fall back to `<body>` when those are missing.
- Collapse repeated whitespace and write each page as a UTF-8 text file with a URL/Title header.
- Maintain a JSON index (`data/raw/_index.json`) mapping URLs to filenames for later citation.
- Insert a one-second delay between requests to avoid hammering the server.

The URL Prioritization Bug

My first implementation sorted all sitemap URLs alphabetically and took the first 25:

```
urls = sorted(set(urls))[:MAX_PAGES]
```

This silently produced a dataset of 24 blog posts and 1 homepage — and no product pages. Because `/blog/*` URLs sort alphabetically before `/pricing`, `/solutions/apparel`, `/solutions/automotive`, etc., the cap excluded every page a real Vizcom user would want to ask about. The bug surfaced only when I tested the chatbot with the query "Can I use Vizcom for apparel?" and watched it confidently fail.

The fix was a small URL-prioritization function that bins URLs into three categories — priority (pricing, about, solutions, careers, etc.), other, and blog — and fills the page cap from priority first. I also raised the cap from 25 to 40 to give the model fuller coverage.

Lesson: when capping a dataset, the cap is a content choice, not just a performance choice. Alphabetical sort is a default that quietly biases the data.

Output

Forty pages scraped, including: homepage, pricing, enterprise, careers, contact, data security, and dedicated solutions pages for apparel, automotive, footwear, gaming, and industrial design. The remainder are blog posts and university (educational) content.

4.2 Phase 2 — Synthetic Q&A Generation

The `qa_generator` module reads each cleaned page, sends it to Llama 3.3 70B with a structured prompt, and asks for exactly eight diverse question-answer pairs grounded in the page content.

Prompt Strategy

The prompt is explicit about three things: that the answer must be grounded in the provided content (no hallucination), that the model should produce a mix of question types (factual, how-to, capability, comparative), and that responses must be valid JSON to enable reliable parsing.

```
response_format={"type": "json_object"}
```

This Groq feature constrains the model output to valid JSON, eliminating the need for fragile post-hoc parsing. Combined with a retry loop on JSON errors and rate-limit handling, the generator processed all 40 pages successfully without manual intervention.

Output

320 (question, answer, source_page, source_title) tuples saved as `data/qa_dataset.csv`. Spot-checking confirmed the questions are natural and diverse - they include things like "What is Vizcom for Apparel Design?", "How does Vizcom help automotive teams maintain design intent from sketch to production?", and "Is Vizcom SOC 2 compliant?"

4.3 Phase 3 — Vector Database Construction

The `vector_store` module builds two persistent Chroma collections from the scraped data and the synthetic Q&A dataset.

Two Collections, One Purpose

- **vizcom_docs.** This collection stores chunks of the raw scraped pages. Documents are split with LangChain's `RecursiveCharacterTextSplitter` using a chunk size of 800 characters and 100-character overlap. The splitter prefers paragraph boundaries, then sentence boundaries, then word boundaries — preserving semantic units wherever possible. The collection holds 244 chunks across 40 source pages.
- **vizcom_qa.** This collection embeds the questions (not the answers) from each of the 320 Q&A pairs. The metadata for each entry carries the curated answer, the source URL, and the source page title. Searching this collection at query time is effectively a fast "have I seen a question like this before?" lookup.

Why two collections matter

Embedding the questions (rather than the answers or full pages) gives the Q&A collection a sharply different shape than the docs collection. A user query like "How much is Vizcom?" lands close to the question embedding for "How much does Vizcom cost?" but not particularly close to any chunk of marketing prose about pricing. This separation is what makes the hybrid retrieval architecture meaningful — the two collections serve different retrieval semantics.

4.4 Phase 4 — Hybrid Retrieval Logic

This phase is the intellectual core of the project. The retriever (`src/retriever.py`) and the chatbot orchestrator (`src/chatbot.py`) together implement a three-mode retrieval strategy.

Three modes, two thresholds

- **Mode 1 - Direct Q&A match.** Search the `vizcom_qa` collection. If the top match has a cosine distance below 0.40, return that pre-written answer directly. No LLM call is needed. This mode is fast, deterministic, and grounded in curated content.
- **Mode 2 - Generated from documents.** If no Q&A pair is close enough, search `vizcom_docs`. If the top chunk has a distance below 1.00, retrieve the top four chunks and pass them as context to Llama 3.3 70B to synthesise an answer. The synthesised answer includes source citations.
- **Mode 3 - Out of knowledge base.** If even the vector search returns a distance above 1.00, return a polite refusal: "I don't have specific information about that in my Vizcom knowledge base." This prevents the chatbot from hallucinating on questions truly outside its domain.

Conversation Memory and Query Rewriting

The naive implementation passed each user query directly to the embedder. Testing surfaced a serious limitation: follow-up questions using pronouns failed dramatically. After asking "What is Vizcom?" and then "Can I use it for free?", the second query landed at distance 1.115 — well above the decline threshold — because the embedding of "it" carries no information about Vizcom. The chatbot would politely decline to answer a question for which we had a perfect pre-written answer.

The fix is a small LLM-based query rewriter that runs before retrieval whenever (a) conversation history exists and (b) the query contains an ambiguous pronoun (it, that, this, these, those, them, they). The rewriter prompts Llama to produce a standalone version of the question using recent history.

After the rewrite: "Can I use it for free?" becomes "Can I use Vizcom for free?" and the distance drops from 1.115 (declined) to 0.004 (direct match). Same conceptual question, dramatically different result.

Beyond rewriting, the LLM generation step (Mode 2) also receives a sliding window of the last four conversation turns as context, allowing it to maintain coherence across multi-turn exchanges.

4.5 Phase 5 — Streamlit User Interface

The Streamlit application (`app.py`) wraps the chatbot in a clean web UI. It uses session state for conversation history, `@st.cache_resource` to load the embedding model once per session, and `@st.cache_data` for the project statistics shown in the sidebar.

UI Features

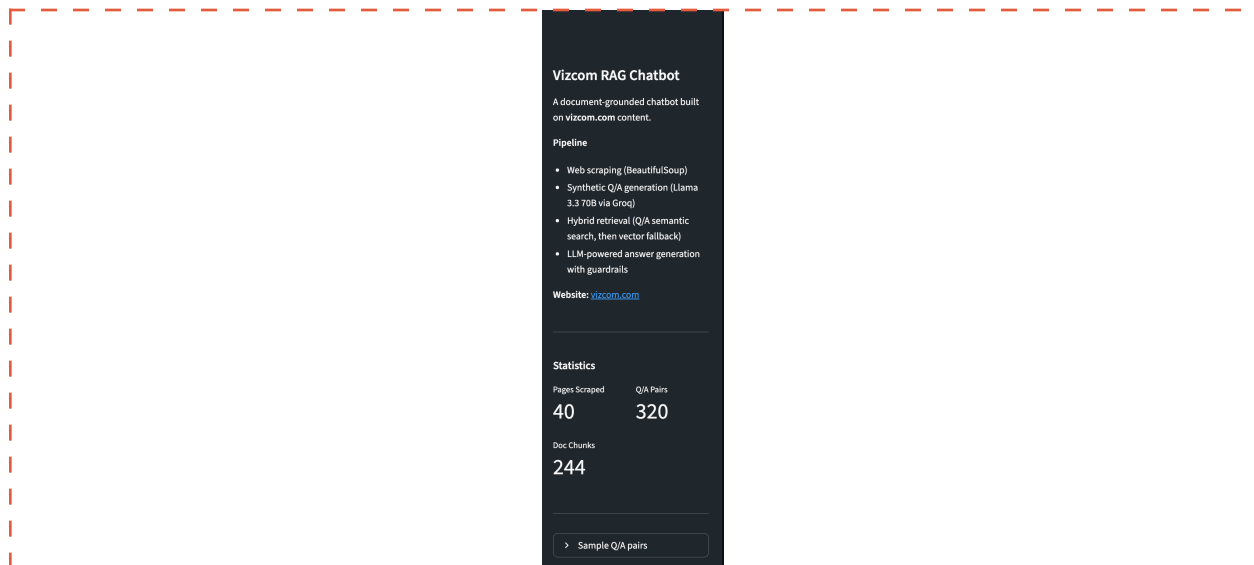
- Conversational chat interface with multi-turn history.
- Mode badge under each answer showing whether it was a direct Q&A match, generated from documents, or out of knowledge base.
- Confidence score (cosine distance) for every answer, so the user can see how certain the system is.
- Expandable Sources panel listing every source URL contributing to the answer, with clickable links to the original Vizcom pages.
- Sidebar with project description, pipeline overview, live statistics (pages scraped, Q&A pairs, doc chunks), and a sample of randomly selected Q&A pairs.
- Clear Chat button that resets both the visible message list and the underlying conversation memory.

5. Testing and Results

After completing the rebuild that included the solutions pages, I tested the chatbot against a curated set of twenty queries spanning all three modes plus a multi-turn conversation. The results below show four representative cases. Confidence scores are cosine distances — lower is better.

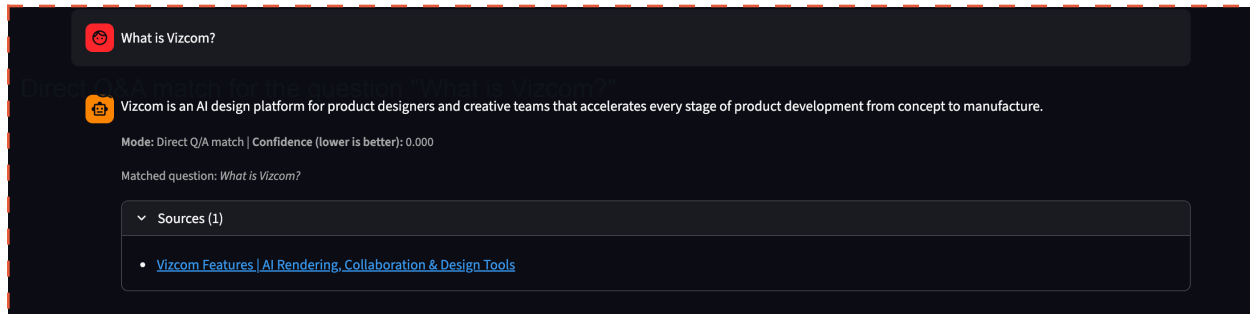
5.1 Sidebar Statistics

The Streamlit sidebar surfaces the size of the underlying corpus and Q&A dataset, giving any user immediate context about what the chatbot knows.



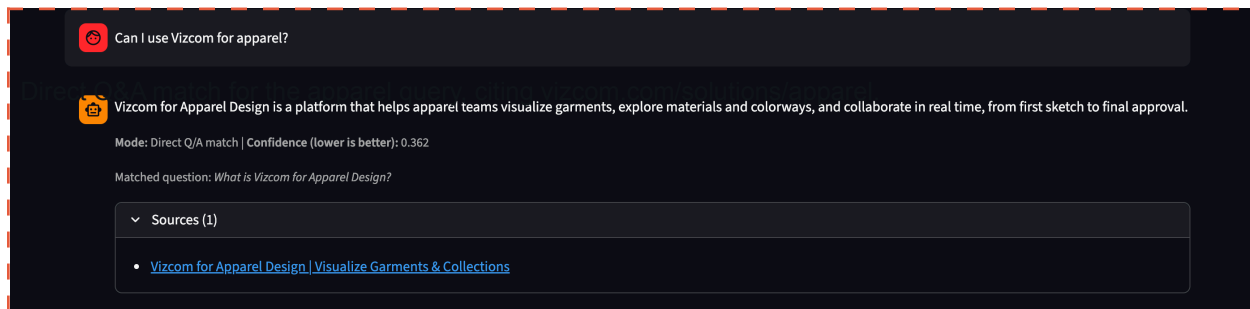
5.2 Mode 1 — Direct Q&A Match

"What is Vizcom?" routes to the Q&A collection with a confidence of essentially zero — the user query nearly perfectly matches the pre-generated question. The pre-written answer is returned in milliseconds with no LLM call.



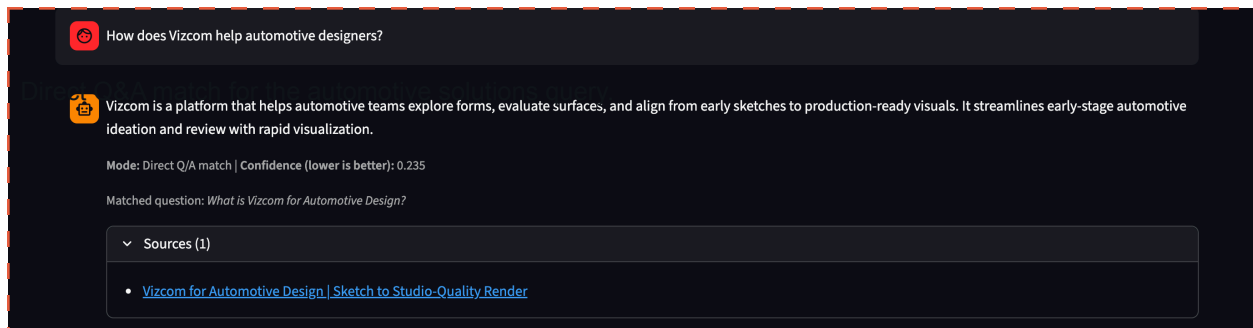
5.3 Mode 1 — Solutions Coverage

"Can I use Vizcom for apparel?" correctly resolves to a direct match against the apparel solutions page. The Sources panel cites vizcom.com/solutions/apparel, demonstrating that the URL prioritisation fix paid off.



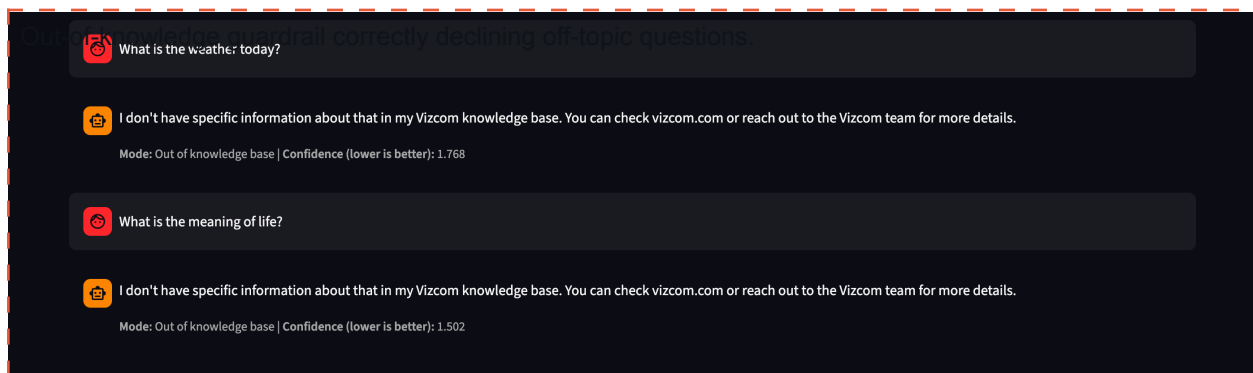
5.4 Mode 1 — Domain-Specific Match

"How does Vizcom help automotive designers?" matches the Vizcom for Automotive Design Q&A pair with low distance. The response is concise and grounded in the source page.



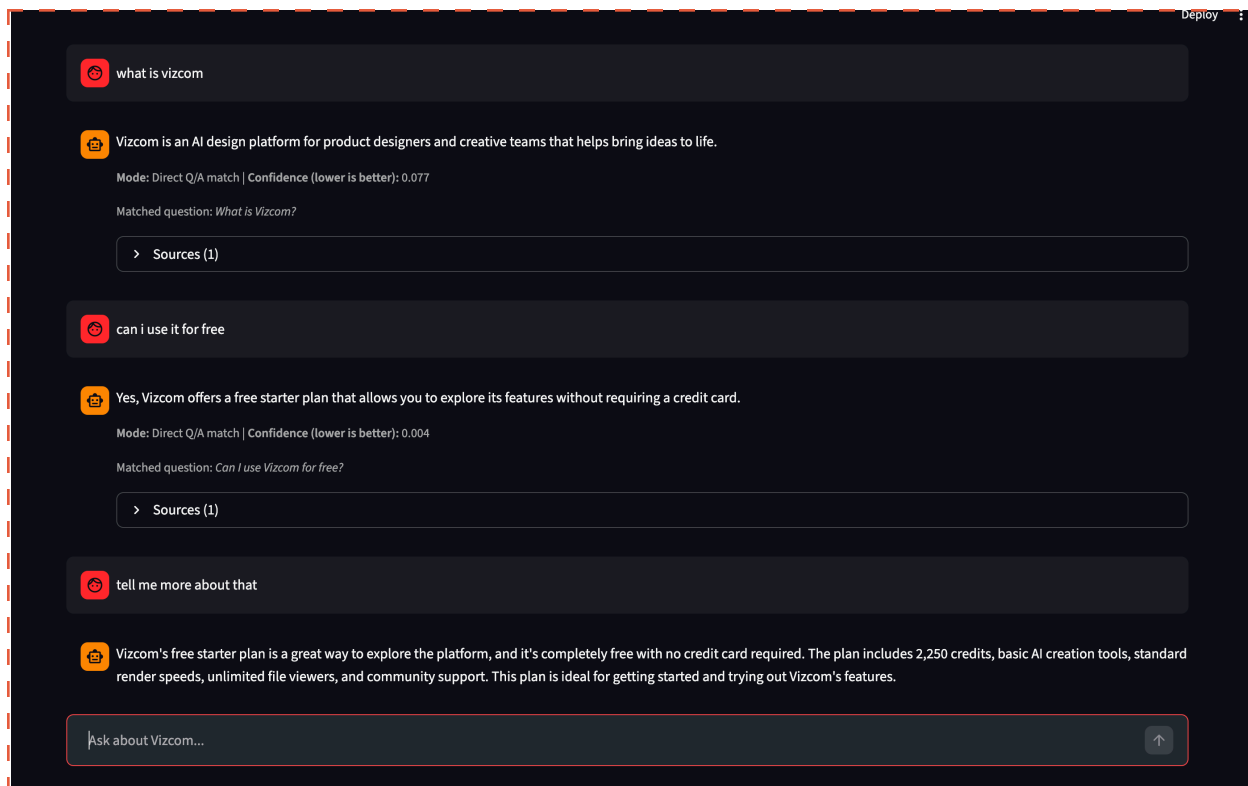
5.5 Mode 3 — Out-of-Knowledge Guardrail

Off-topic queries are routed to the decline response with high distance scores. "What is the weather today?" (distance 1.768) and "What is the meaning of life?" (distance 1.502) both return the polite refusal rather than hallucinating an answer from irrelevant chunks.



5.6 Multi-Turn Conversation with Query Rewriting

A three-turn conversation demonstrates conversation memory and pronoun resolution working together. Turn 1 hits the Q&A collection directly. Turn 2 ("Can I use it for free?") is rewritten internally to include Vizcom, then matches the free-plan Q&A pair with distance 0.004. Turn 3 ("Tell me more about that") falls through to vector mode, where the LLM uses retrieved chunks plus conversation history to produce a detailed answer including the specific 2,250-credit allocation.



Summary of Behavior

- All major user-facing questions about Vizcom's products and solutions return correct, source-cited answers.
- Off-topic and unanswerable questions are declined politely with no hallucination.
- Multi-turn conversations resolve pronouns and maintain context across at least three turns.
- Every answer includes a confidence score and an expandable list of source URLs.

6. Limitations and Future Improvements

Limitations

Knowledge cutoff at scrape time

The chatbot only knows what was on vizcom.com at the moment of the scrape. New blog posts, product launches, or pricing changes published after the scrape are invisible until the pipeline is re-run. A production deployment would need a scheduled re-scrape (daily or weekly) to stay current.

Marketing-page content is thin

Some of Vizcom's most visually compelling pages (the homepage, certain solutions pages) are image-heavy and contain relatively little extractable text. This produces fragmented chunks like "Make it real / Loading / Get Started / Scroll" that are not particularly useful for retrieval. A more sophisticated scraper would extract structured content (headings, feature descriptions) from the page's underlying React/Next.js data rather than the rendered DOM.

Threshold tuning is manual

The two distance thresholds (0.40 for Q&A match, 1.00 for vector decline) were chosen by inspection of test queries. A more rigorous approach would calibrate them on a labelled validation set, optimizing for precision-recall trade-offs the product team actually cares about.

No fact-checking layer

In Mode 2 (LLM generation from retrieved chunks), the model can subtly hallucinate details even when grounded in real context. There is no automated check that the generated answer is actually supported by the cited sources. A second-pass verification model could catch contradictions before showing answers to users.

Single-language, single-domain

The system only handles English queries and only knows about Vizcom. Cross-domain or multilingual generalization is out of scope.

No user-level personalization

All users see the same retrieval behavior. A real internal Vizcom tool would benefit from user-aware retrieval: sales reps need different content than designers, who need different content than enterprise procurement leads.

Future Improvements

Re-ranking with a cross-encoder

After initial vector retrieval, a cross-encoder model (such as BAAI/bge-reranker) could reorder the top 20 candidates down to the best 3-5 before they reach the LLM. This is a well-known

production technique that typically improves answer quality more than larger embedding models do.

Hybrid keyword + semantic search

BM25 keyword retrieval over the raw chunks, combined with semantic retrieval and reranking, would catch queries that contain proper nouns or product names where embeddings can miss exact matches. LangChain's EnsembleRetriever or a custom combination would integrate cleanly.

User feedback loop

Thumbs up / thumbs down on each answer, with the feedback fed back into the Q&A dataset (good answers become new curated pairs; bad answers flag chunks to be reviewed). Over time this turns the chatbot into a continually improving system rather than a static one.

Fashion-tech specialization

Given my personal interest in fashion technology, a natural extension would be adding fashion-specific embeddings and a fashion-domain knowledge graph (materials, construction techniques, sustainable manufacturing terminology) so that queries from apparel designers receive deeper, more domain-aware answers.

Deploy as an internal Vizcom tool

With cleaned-up auth, telemetry, and integration with Vizcom's actual help-centre infrastructure, this prototype could become an internal sales-support and customer-success assistant. The next step would be a conversation with the Vizcom engineering team about the constraints (data residency, latency budgets, brand voice guidelines) that a production version would need to respect.

7. Key Learnings

Technical Skills Developed

- End-to-end RAG pipeline construction: scraping, chunking, embedding, retrieval, generation.
- Practical use of LangChain for text splitting and orchestration.
- Working with vector databases (Chroma's persistent client API).
- Prompt engineering for structured JSON output with reliability constraints.
- Streamlit application development with session state, caching, and chat widgets.
- API integration with Groq, including rate-limit handling and retry logic.
- Environment management with venv, .env secrets, and .gitignore hygiene.

Conceptual Lessons

RAG vs fine-tuning

Before this project I treated "add my company's data to an LLM" as a single problem. Building the system clarified that RAG and fine-tuning solve different problems: RAG injects facts at query time and is the right tool when documents change frequently and citations matter; fine-tuning modifies model weights and is the right tool when you want to change behavior, tone, or format. For a Vizcom-content chatbot, RAG is unambiguously the correct choice.

Retrieval is hard

The retrieval step is where most of the practical engineering effort lives. Two of the most important bugs in the project (URL prioritization; pronoun-induced retrieval failure) were retrieval bugs, not generation bugs. The model is rarely the limiting factor; the quality and shape of what you feed it is.

Confidence scores are first-class outputs

Exposing distance scores in the UI (rather than burying them in logs) made the chatbot feel honest. Users can see at a glance whether the system is certain ("Direct Q&A match, confidence 0.004") or guessing ("Generated from documents, confidence 0.85"). That transparency is a meaningful product feature, not just a debugging affordance.

Defaults are content choices

The alphabetical-sort bug taught me that any cap or limit silently makes a content choice. The default "first N alphabetical" sounds neutral but is anything but. This is the kind of decision that needs to be deliberate, especially when building tools that present themselves as comprehensive.

Working with Vizcom Content

The depth of content on vizcom.com surprised me even though I work there. The university section in particular (Vizcom 101, automotive design education, design challenges) is a genuinely useful resource for designers learning the platform. Building this chatbot was, in a small way, a forcing function to read it all properly — which I suspect is the kind of side benefit any internal-tool project produces.

8. Conclusion

The project delivers a working end-to-end RAG chatbot grounded in vizcom.com content, with all five required phases implemented plus several enhancements (LLM query rewriting, three-mode retrieval, confidence-visible UI). It demonstrates not just that the techniques work, but that they cohere into a system whose behavior can be reasoned about and improved iteratively.

More personally, the project clarified what is now possible for non-engineering teammates: I could not have built this two weeks ago, but the combination of clear documentation, fast inference platforms like Groq, and well-designed libraries like LangChain and Streamlit means that a product-focused person on a small team can credibly prototype AI features. That capability matters at Vizcom, where the gap between "someone has an idea" and "engineering has bandwidth" is the rate-limit on a lot of useful internal tooling. I am leaving this project intending to actually use what I built — possibly extending it to a Slack-integrated internal helper, or to a fashion-tech-specialized variant that explores the intersection I care about most.

If a product expert can build this in two weeks, the bar for internal AI tooling has just shifted. The bottleneck is now imagination and clear product thinking — not engineering bandwidth.

Appendix: Repository Structure

agai03-rag-vizcom/

```
├─ data/

|   └─ raw/                    # 40 scraped page text files

|       └─ _index.json        # URL <-> filename map for citations

|       └─ qa_dataset.csv     # 320 synthetic Q&A pairs

├─ src/

|   └─ scraper.py             # Phase 1: web scraping

|   └─ qa_generator.py        # Phase 2: Q&A generation

|   └─ vector_store.py        # Phase 3: Chroma collections

|   └─ retriever.py           # Phase 4a: hybrid retrieval

|       └─ chatbot.py         # Phase 4b: orchestrator + query rewriting

├─ chroma_db/                 # persistent vector store

├─ app.py                     # Phase 5: Streamlit UI

├─ requirements.txt

├─ README.md

├─ .env                       # GROQ_API_KEY (gitignored)

└─ .gitignore
```

Repository link

Code, documentation, and screenshots are available at: github.com/jandavdyk

Acknowledgements

Thanks to the AGAI-03 instructor and cohort for the original assignment specification, to the Vizcom team for building a product that is fun to work with, and to the open-source communities behind LangChain, Chroma, sentence-transformers, and Streamlit.